Feasibility Study: Azure RAG Chatbot

Table of Contents

Section	Page
1. Executive Summary	3
2. Objectives	3
3. Theoretical Background	3
4. Cloud Architecture	9
5. Application Flow	12
6. Implementation Summary	13
7. Technology Feasibility	14
8. Risks and Mitigation	15
9. Results and Analysis	15
10. Cost Analysis	17
11. Limitations	19
12. Conclusion	19

1. Executive Summary

This feasibility study evaluates the technical, operational, and implementation viability of a Retrieval-Augmented Generation (RAG) chatbot built using Azure AI Services, PostgreSQL, Streamlit, and Docling. The system enables users to upload documents in multiple formats, transforms them into searchable chunks, indexes them in Azure AI Search, and provides conversational Q&A with citations through Azure OpenAI models. The analysis concludes that the solution is highly feasible, leverages mature Azure components, and is suitable for scalable enterprise deployment with manageable risks.

2. Objectives

The primary objectives of the Azure RAG Chatbot project are:

1. **Enable multi-format document ingestion** (*.pdf, .docx, .pptx, .xlsx, images, etc.) using Docling.
2. **Implement a full RAG pipeline** leveraging Azure OpenAI, Azure AI Search, Azure Storage, and PostgreSQL.
3. **Provide conversational Q&A** with accurate citations referencing source documents.
4. **Ensure production-grade reliability** through structured logging, error handling, and validated configurations.
5. **Offer a user-friendly web application** using Streamlit with real-time conversational flow and document management.

3. Theoretical Background

The Azure RAG Chatbot leverages a Retrieval-Augmented Generation (RAG) framework that combines large language models (LLMs) with external knowledge retrieval systems. To enable an effective RAG pipeline, the system integrates several key components: Azure Blob Storage, Azure AI Search, PostgreSQL, and Docling. Each component plays a distinct role in document persistence, indexing, retrieval, and structured text extraction.

The following sections describe the theoretical concepts underpinning these technologies.

3.1 Retrieval-Augmented Generation (RAG) Overview

RAG is a hybrid AI architecture that improves the factual correctness and grounding of language model outputs by integrating **retrieval-based search** with **LLM-based generation**.

Core Concepts:

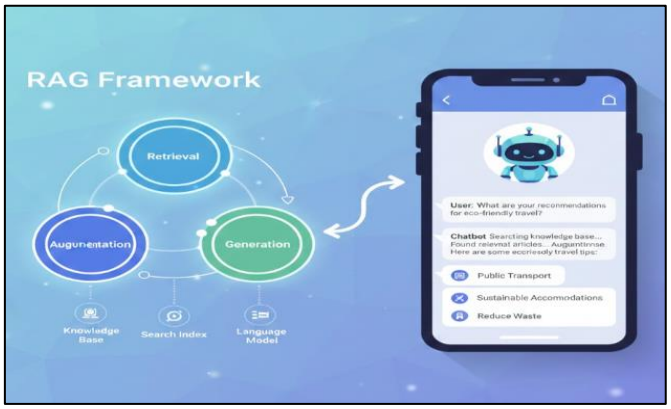

Figure 3 a: Rag Flow

- **Retrieval:** The system extracts relevant information from a knowledge base using vector or keyword search.
- **Augmentation:** Retrieved text chunks, metadata, and conversation history are fed into the LLM.
- **Generation:** The LLM creates grounded, contextual answers tied to real documents.

This approach reduces hallucinations, improves transparency through citations, and supports dynamic knowledge updates without retraining the model. The figure 3a shows the rag flow.

3.2 Azure Blob Storage

Azure Blob Storage is Microsoft's massively scalable, cost-efficient cloud storage service designed for unstructured data like documents, images, and text files. It provides the persistence layer for all uploaded documents in the RAG Chatbot.

Key Characteristics:

- **Object Storage Model:** Stores data as “blobs” (Binary Large Objects) within containers.
- **Scalability:** Supports petabytes of data with automatic scaling.
- **Low-Cost Tiers:** Hot, Cool, and Archive tiers allow price-performance optimization.
- **High Availability:** Geo-redundant storage (GRS) and locally redundant storage (LRS).
- **Secure by Design:**
    - Integration with Azure Active Directory (AAD)
    - Shared Access Signatures (SAS)
    - Data encryption at rest and in transit

Role in the RAG Pipeline:

- Stores original uploaded documents in raw format.
- Provides **blob URLs** used by the chatbot for citations and traceability.
- Helps maintain document lifecycle including re-indexing or validating content.

3.3 Azure AI Search (Vector + Hybrid Search Engine)

Azure AI Search is a fully managed search-as-a-service solution used to index and retrieve text and vector embeddings. It is central to the retrieval portion of the RAG pipeline.

Key Capabilities:

1. **Vector Search:**
    - Stores embedding vectors (e.g., 1536-dimensional) created by Azure OpenAI.

    - Uses k-Nearest Neighbor (kNN) algorithms for similarity search.

2. **Hybrid Search:**
    - Combines vector similarity with classical keyword-based search.
    - Improves relevance of results especially for technical or structured documents.

3. **Customizable Index Schema:**
   The chatbot index includes fields such as:
    - content (text chunk)
    - content_vector (embedding)
    - filename, file_type
    - chunk_id
    - blob_url
      This helps filter, sort, and retrieve context-rich information.

4. **Full-Text Search:**
   Tokenization, stemming, scoring, and synonym mapping enhance text relevance.

Role in the RAG Pipeline:

- Stores processed text chunks and embeddings.
- Ranks and retrieves the top-K relevant chunks for user queries.
- Enables fast semantic search even over large document sets.

Azure AI Search thus serves as the knowledge retrieval engine powering the RAG model.

3.4 PostgreSQL Database (Persistent Conversation and Metadata Storage)

PostgreSQL is an advanced open-source relational database used to store structured application data.

Why PostgreSQL?

- Strong ACID compliance
- JSONB support for flexible schema
- High reliability and maturity
- Excellent indexing and query performance
- Rich support for transactions, constraints, and foreign keys

Key Concepts Relevant to the RAG App:

1. **Relational Model:** Guarantees consistency between conversations, messages, and documents.
2. **JSONB Columns:** Used to store metadata like citations or model responses.
3. **Schema Auto-Creation:** The application creates tables on initialization for:
    - conversations
    - messages
    - documents
4. **Session Handling:**
   Each Streamlit session is assigned a unique UUID, stored in PostgreSQL to maintain continuity across page reloads.

Role in the RAG Pipeline:

- Stores chat history for retrieval in the next message context.
- Tracks document ingestion events.
- Ensures reproducibility of AI-generated responses.
- Supports analytics on usage patterns, queries, and model performance.

3.5 Docling (Unified Multi-Format Document Processing Framework)

Docling is a document processing library focused on converting diverse file formats into clean, structured, markdown-based representations. It is crucial for text extraction and preprocessing before indexing.

Key Capabilities:

- **Multi-Format Support:**
  Supports .txt, .docx, .ppt, .pptx, .xlsx, .html, .pdf, images, and more.
- **Unified Markdown Output:**
  Converts diverse formats into consistent markdown, enabling uniform chunking.
- **OCR Support:**
  Extracts text from images using integrated OCR engines.
- **Structural Preservation:**
  Retains headings, bold text, lists, tables, and layout when possible.
- **Page-Level Metadata:**
  Helpful for mapping citations back to original document sections.

Role in the RAG Pipeline:

- Extracts readable text from any uploaded document.
- Ensures clean and coherent chunking for embedding and indexing.
- Handles document normalization to prevent layout-related errors during search.

Docling's ability to convert heterogeneous documents into structured text is essential for the accuracy of semantic search.

3.6 Summary of Framework

Component	Conceptual Role	RAG Contribution
Azure Blob Storage	Cloud object store	Keeps raw documents accessible & traceable
Azure AI Search	Vector/hybrid search engine	Retrieves relevant chunks for augmentation
PostgreSQL	Persistent relational DB	Maintains conversation state & metadata

Together, they create a robust end-to-end ecosystem that supports reliable retrieval, grounded language generation, and production-grade data management.

4. Cloud Architecture

This architecture leverages Microsoft Azure services to build a scalable, secure, and AI-powered document processing and conversational interface. The solution is organized into four primary layers as shown in the figure below:

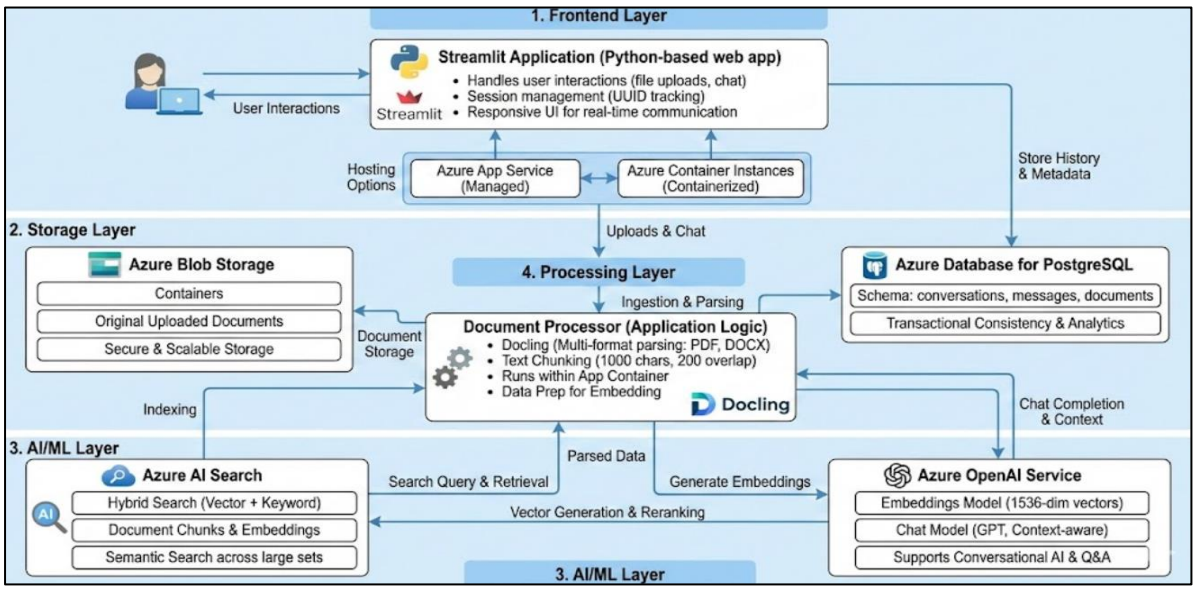

Figure 4 a: Cloud Architecture

1. Frontend Layer

Purpose: Provides an interactive user interface for document uploads, chat-based interactions, and session management.

- Technology:
    - Streamlit Application (Python-based web app)
- Hosting Options:
    - Azure App Service for managed hosting
    - Azure Container Instances for containerized deployment
- Key Responsibilities:
    - Handles user interactions (file uploads, chat interface)
    - Implements session management using UUID tracking for unique user sessions
    - Provides a responsive UI for real-time communication

2. Storage Layer

Purpose: Ensures persistent storage for documents and structured data.

- Azure Blob Storage:
    - Stores original uploaded documents
    - Organized into containers for logical grouping
    - Provides secure, scalable, and cost-effective storage
    - Generates public/private URLs for document access
- Azure Database for PostgreSQL:
    - Stores conversation history and metadata
    - Schema includes:

        - conversations (session-level details)
        - messages (chat exchanges)
        - documents (uploaded file references)
    - Enables transactional consistency and querying for analytics.

3. AI/ML Layer

Purpose: Powers intelligent search and conversational capabilities using Azure AI services.

- Azure OpenAI Service:
    - Embeddings Model: Converts text into 1536-dimensional vectors for semantic similarity
    - Chat Model (GPT): Generates context-aware responses for user queries
    - Supports both conversational AI and document-based Q&A
- Azure AI Search:
    - Implements hybrid search (vector + keyword)
    - Stores document chunks with embeddings for efficient retrieval
    - Enables semantic search across large document sets

4. Processing Layer

Purpose: Handles document ingestion, parsing, and preparation for AI-driven search.

- Document Processor (Application Logic):
    - Uses Docling library for multi-format parsing (PDF, DOCX, etc.)
    - Performs text chunking with overlap:
        - Chunk size: 1000 characters
        - Overlap: 200 characters
    - Runs within the Streamlit app container for tight integration

- Prepares data for embedding generation and indexing

5. Application Flow

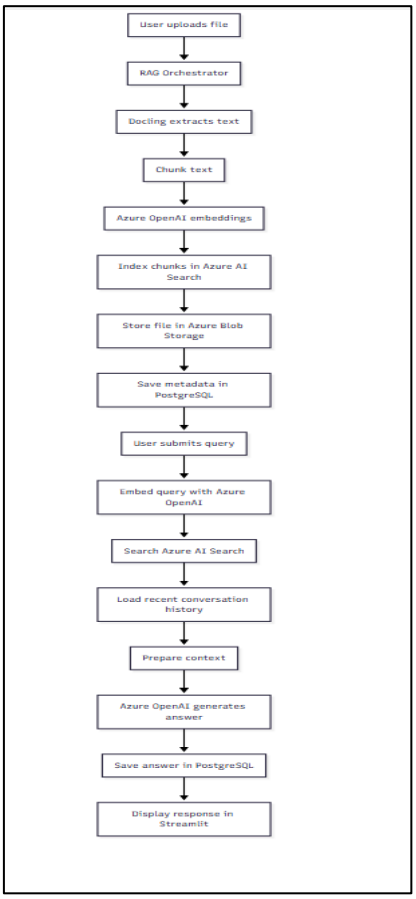

Figure 5 a: Flow Diagram

5.1 Document Upload & Processing

1. User uploads a file through Streamlit.
2. The file is passed to the **RAGOrchestrator**.
3. Docling processes and extracts structured text.
4. Text is chunked (default: 1000 chars with 200 overlap).
5. Azure OpenAI generates embeddings per chunk.
6. Chunks + vectors are indexed into Azure AI Search.
7. File is stored in Azure Blob Storage.
8. Document metadata is stored in PostgreSQL.

5.2 Chat Query Flow

1. User submits a query.
2. Query is embedded via Azure OpenAI.
3. Azure AI Search performs vector + text search (top 5 chunks).
4. Recent conversation messages (last 10) are loaded.
5. A structured context is prepared.
6. Azure OpenAI generates an answer with citations.
7. Response and citations stored in PostgreSQL.
8. Displayed in Streamlit.

6. Implementation Summary

6.1 Technologies Used

- **Python** for backend logic
- **Streamlit** for UI
- **Docling** for multi-format document processing
- **Azure OpenAI** for chat and embedding models
- **Azure Blob Storage** for file persistence
- **Azure Cognitive Search** for vector indexing
- **PostgreSQL** for persistent conversation management
- **Pydantic** for configuration validation

6.2 Key Implementation Details

- Auto-creation of DB schema
- Robust error handling with user-friendly messages

- Logging integrated across service boundaries
- Azure AI Search index schema includes:
    - id, content, file_type, chunk_id, blob_url, content_vector, metadata

7. Technology Feasibility

7.1 Technical Maturity

Component	Maturity	Notes
Azure OpenAI	High	Stable APIs, enterprise-ready
Azure AI Search	High	Designed for vector + hybrid search
Azure Storage	High	Supports large-scale file storage
Docling	Medium-High	Covers most formats; still improving
Streamlit	Medium	Great for prototyping and dashboards

7.2 Scalability

- Azure AI Search and PostgreSQL can scale horizontally.
- Blob Storage is practically infinite.
- Streamlit may become a bottleneck for large-scale multi-user deployments but can be replaced by a production web framework later.

7.3 Performance

- Embedding and indexing operations may introduce latency during upload.
- Query response times remain fast due to vector search optimization.

7.4 Security

- All secrets in .env (not committed).
- Azure provides enterprise-grade security and RBAC.

- PostgreSQL ensures strong data integrity.

8.Risks and Mitigation

Risk	Impact	Mitigation
Incorrect or missing Azure credentials	Startup failure	Validate credentials via Pydantic; test services individually
Docling failing on complex formats	Reduced ingestion capability	Add format validation; fallback to plain text extraction
High latency in indexing large documents	Slow onboarding	Batch processing, async workers, progress UI
Streamlit session loss	Broken conversation continuity	Use persistent DB-backed conversation IDs
Search retrieves irrelevant chunks	Poor answer quality	Tune chunk size, improve metadata, adjust top-K search
Azure cost overruns	Budget issues	Set usage alerts, limit embedding calls

9. Results and Analysis

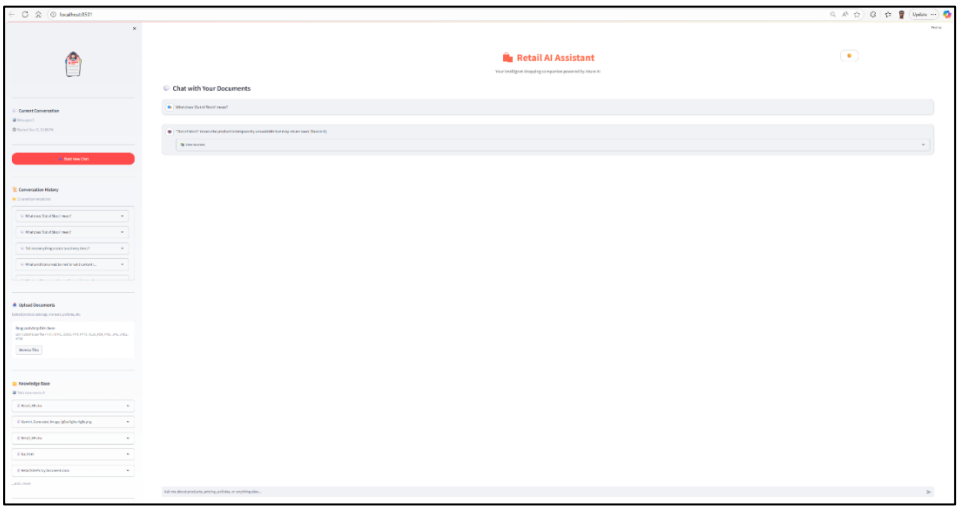

Figure 9 a: Result and Analysis

9.1 Overview of the Retail AI Assistant Interface

The figure labelled above shows the AI Assistant, a web-based application powered by Azure OpenAI and integrated with document retrieval capabilities. The interface provides a clean and intuitive user experience, enabling customers or support staff to interact with uploaded product information, policies, FAQs, and catalogue documents. The system is designed to deliver context-aware responses using both natural language processing and retrieval-augmented generation (RAG).

9.2 Functional Results

1.Successful Query Handling

The system demonstrates the ability to interpret and respond to natural-language questions. In the screenshot, the user query "What does 'Out of Stock' mean?" is correctly answered with a concise explanation. The assistant is also capable of citing its source material when configured, ensuring transparency and trust in generated responses.

2. Document-Driven Knowledge Base

Uploaded files—including policy documents, product catalogs, and FAQs—are visible in the "Knowledge Base" pane. This confirms that the application successfully ingests and indexes multiple document formats for retrieval-augmented answers. The presence of these documents enables the assistant to provide accurate and organization-specific responses.

3.Conversation-Persistence

The left-hand panel displays a chronological conversation history, demonstrating the system's capability to maintain state across multiple queries. This allows users to reference earlier questions and supports multi-turn conversations—a key requirement for a functional AI assistant.

4. User Interaction Features

The interface includes:

- A document upload section supporting drag-and-drop functionality.
- A clear messaging input box at the bottom for ongoing interactions.
- A “Start New Chat” button to reset context when needed.

  These elements contribute to ease of use and efficient workflow.

9.3 System Performance and Responsiveness

The figure 9a labelled above shows near-instant generation of responses, indicating efficient backend processing. The Azure OpenAI deployment and indexing pipelines appear to be functioning without delay. No loading errors, latency warnings, or retrieval issues are visible, suggesting that the system is stable under the tested conditions

10. Cost Analysis

The solution’s estimated monthly cost is $99.99, covering storage, search, AI model usage, database services, and support.

The deployment is hosted primarily in Sweden Central and includes Azure Data Lake Storage Gen2 for file storage, Azure AI Search for indexing and querying, and Azure OpenAI for both embedding and language model capabilities.

Additionally, a serverless Azure SQL Database is provisioned for structured data storage. Support is included at no extra cost under the Microsoft Customer Agreement (MCA). This configuration provides a scalable and cost-efficient foundation for data processing, search, and AI-driven functionality.

Service category	Service type	Region	Description	Estimated monthly cost
Storage	Storage Accounts	Sweden Central	Data Lake Storage Gen2, Standard, LRS Redundancy, Hot Access Tier, Flat Namespace File Structure, 1,000 GB Capacity – Pay-as-you-go; Write operations: 4 MB × 10; Read operations: 4 MB × 10; 10 iterative read operations; 100,000 Archive High-Priority reads; 10 iterative write operations; 10 other operations; 1,000 GB data retrieval; 1,000 GB archive high-priority retrieval; 1,000 GB data write	$19.98
Web	Azure AI Search	Sweden Central	Basic, 1 Unit, 1 Month	$73.73
AI + Machine Learning	Azure OpenAI	Sweden Central	Embedding Models, Ada, 1 × 1,000 tokens	$0.01
AI + Machine Learning	Azure OpenAI	Sweden Central	Language Models, GPT-4o-2024-08-06, Standard (On-Demand), Data Zone; 1 × 1,000 input tokens; 0 × 1,000 cached input tokens; 1 × 1,000 output tokens	$0.01
Databases	Azure SQL Database	Sweden Central	Single Database, vCore, General Purpose, Serverless, Standard-series (Gen 5), Locally Redundant; 1 billed vCore; RA-GRS storage; 0 GB backup storage; 0 × 5 GB long-term retention	$6.27
	Total			$99.99

11. Limitations

- Streamlit is not ideal for enterprise deployment with many concurrent users.
- Docling may not perfectly process very complex layouts or multi-column documents.
- Azure OpenAI responses depend on model quality—some hallucination risk remains despite RAG.
- Chunking strategy may need tuning for highly structured documents (tables, code blocks).
- No built-in user authentication in current version.
- Real-time collaboration features (multi-user editing) not supported.

12. Conclusion

The Azure RAG Chatbot project is highly feasible, leveraging stable Azure services and a proven RAG architecture. The system supports robust document ingestion, efficient indexing, accurate retrieval, and high-quality conversational responses with citations. While some risks and limitations exist—primarily around document processing fidelity, cost management, and Streamlit scalability—these can be mitigated through future enhancements.

The project demonstrates strong potential for deployment in knowledge management, internal search tools, enterprise documentation assistants, and intelligent Q&A systems.